11. LoRA & PEFT Fine-tuning

Overview

Full fine-tuning updates every weight; LoRA and other PEFT methods inject low-rank adapters into attention projections. You will see where adapters attach, how many trainable parameters remain, and memory savings on a local GPU when fine-tuning transformers from Hugging Face.

Lab: Lab 11.

Prior modules: Module 09, **Module 10**.

Contents

Module track

This module builds on **Module 09**, **Module 10**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

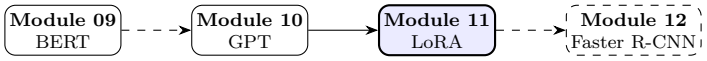

Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Fine-Tuning at Scale

Full fine-tuning of BERT (**Module 09** (BERT architecture)) or GPT (**Module 10**) updates all parameters, expensive for 7B+ models. **Parameter-Efficient Fine-Tuning (PEFT)** adapts small modules instead.

2 LoRA Definition

LoRA [3] injects low-rank updates into frozen weight $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{BA}, \quad \mathbf{B} \in \mathbb{R}^{d\times r},\ \mathbf{A} \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{1}$$

Forward pass: $\mathbf{h} = \mathbf{W}_0\mathbf{x} + \mathbf{BAx}$.

Note. Train only $\mathbf{A}$, $\mathbf{B}$ ($\approx$ 0.1–1% of params); $\mathbf{W}_0$ stays frozen from pre-training.

3 Initialization and Scaling

Initialize $\mathbf{A}$ with Kaiming uniform, $\mathbf{B} = \mathbf{0}$ so $\Delta\mathbf{W} = \mathbf{0}$ at start. Scaling factor α/r controls effective learning rate of the adapter:

$$\mathbf{h} = \mathbf{W}_0\mathbf{x} + \frac{\alpha}{r}\mathbf{BAx}. \tag{2}$$

4 Where to Apply LoRA

Typically target attention projections $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$ in **Module 04**. MLP layers optional; embeddings usually frozen.

Table 1: PEFT method comparison.

Method	Trainable %	Inference overhead
Full fine-tune	100%	None
LoRA	0.1 to 1%	None (mergeable)
Adapter layers	2 to 5%	Extra latency
Prompt tuning	<0.1%	Longer context

5 Merging and Deployment

At inference, $\mathbf{W} = \mathbf{W}_0 + \mathbf{BA}$ can be merged once, no runtime overhead. Multiple task-specific LoRA modules can be swapped without reloading base weights.

6 Rank Selection

Higher r increases capacity but risks overfitting on small datasets.

$$\#\text{LoRA params per layer} = r(d + k). \tag{3}$$

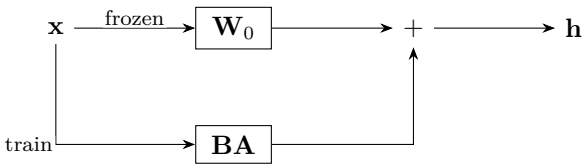

Figure 2: LoRA parallel path (1).

7 Lab

Lab 11 applies LoRA to a transformer for classification; compare GPU memory to full fine-tune in **Lab 09**. XAI on fine-tuned models in **Module 20**.

8 QLoRA and Quantization

4-bit NF4 base weights with LoRA adapters reduce memory further:

$$\mathbf{W} = \text{dequant}(\mathbf{W}_0^{\text{4bit}}) + \mathbf{BA}. \tag{4}$$

Rank-stabilized LoRA scales α with $\sqrt{r}$ rather than r.

8.1 Multi-Adapter Routing

Serve multiple LoRA experts selected by task ID at inference without full model copies.

9 Extended Intuition

Full fine-tuning updates every weight in a pretrained model; for 7B+ parameters that is infeasible on a 1650. LoRA (Low-Rank Adaptation) injects trainable rank-r matrices into selected linear layers while freezing base weights. For a frozen weight $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$, LoRA adds $\Delta\mathbf{W} = \mathbf{BA}$ with $\mathbf{B} \in \mathbb{R}^{d\times r}$, $\mathbf{A} \in \mathbb{R}^{r\times k}$, $r \ll \min(d, k)$.

Forward pass becomes $\mathbf{W}_0\mathbf{x} + \mathbf{BAx}$; only $\mathbf{A}$ and $\mathbf{B}$ receive gradients. Typical targets: query and value projections in attention (`q_proj`, `v_proj`); sometimes all four projections. PEFT library generalizes to adapters, prefix tuning, and IA3; LoRA remains the default for text and diffusion fine-tunes.

Scaling $\Delta\mathbf{W}$ by α/r keeps update magnitude stable when changing rank. Merge adapters into base weights at deployment for zero inference overhead, or keep separate for multi-tenant serving.

10 Numerical Walkthrough

Fine-tune `distilbert-base-uncased` ($d = 768$) with LoRA on attention query/value, rank $r = 8$, $\alpha = 16$, $B = 16$, $T = 128$.

One LoRA pair on $\mathbf{W}_q \in \mathbb{R}^{768\times768}$: params $768 \cdot 8 + 8 \cdot 768 = 12288$ vs 589824 full layer (about 2% per projection). Four projections with LoRA at $r = 8$: roughly 98k trainable vs 66M frozen (exact count depends on target modules).

Training SST-2: lr $= 3 \times 10^{-4}$ on adapter weights (higher than full fine-tune), 3 epochs. Peak VRAM may drop from 3.5 GB full fine-tune to 2.0 GB LoRA on same batch (optimizer states only on adapters).

Stable Diffusion LoRA on 512px faces: rank 4 to 8, train U-Net attention layers, $B = 1$, accumulation 8, lr $= 10^{-4}$, 800 to 2000 steps on 20 to 50 images.

11 PyTorch Implementation Notes

- `peft` library: `LoraConfig(r=8, lora_alpha=16, target_modules=["query", "value"], task_type="SEQ_CLS")`.
- Wrap model: `model = get_peft_model(base, config)`; print `trainable_params` summary.
- Save only adapters: `model.save_pretrained("./lora_out")`; load with `PeftModel.from_pretrained(base, "./lora_out")`.
- Merge for inference: `model.merge_and_unload()` folds LoRA into $\mathbf{W}_0$.
- Do not apply weight decay to frozen base params; optimizer should filter `p.requires_grad`.

```
from peft import LoraConfig, get_peft_model
config = LoraConfig(r=8, lora_alpha=16, target_modules=["q_proj","v_proj"])
model = get_peft_model(base_model, config)
model.print_trainable_parameters()
```

12 Local GPU Constraints

LoRA enables GPT-2 medium or SD 1.5 fine-tunes on 4 GB where full weights fail due to optimizer state. Use rank 4 to 8 first; doubling rank roughly doubles adapter params and optimizer memory.

8-bit base loading (`load_in_8bit=True`) plus LoRA further cuts memory; requires `bitsandbytes` on Windows WSL or Linux. For diffusion, train at 384px if 512px OOM; LoRA quality often survives moderate resolution drops.

13 Debugging Checklist

- No learning: `target_modules` names mismatch model architecture (BERT uses `query`, GPT-2 uses `c_attn` submodules).
- Adapter saved but load fails: base checkpoint version differs from training base.
- Worse than full fine-tune on large data: rank too low or too few train steps; try $r = 16$.
- OOM despite LoRA: still loading full model in fp32; try fp16 base + fp16 adapters.
- Merged model behaves differently: merge order or scaling α/r wrong in custom implementation.

13.1 Rank Selection and Multi-Adapter Serving

Treat rank r as a capacity knob, not a hyperparameter you guess once. Start at $r = 8$ for text classification; if validation loss plateaus above full fine-tune, try $r = 16$ before touching learning rate. For diffusion style LoRA, ranks 4 to 8 often suffice because U-Net attention layers already encode strong priors from pre-training. Initialization matters: PEFT typically sets $\mathbf{A}$ with Kaiming uniform and $\mathbf{B} = \mathbf{0}$ so the initial forward pass equals the frozen base model exactly. That zero start prevents a random adapter from destroying pretrained logits on step 0.

You can stack multiple LoRA adapters on one frozen base for multi-tenant serving: keep $\mathbf{W}_0$ shared, load adapter A for customer 1 and adapter B for customer 2 at request time. Merged weights (`merge_and_unload`) remove runtime adapter overhead but lose hot-swapping; production teams often merge for latency-critical paths and keep separate adapters for experimentation. When fine-tuning both BERT (**Module 09**) and GPT (**Module 10**) in the same codebase, print `named_modules()` once and map `target_modules` per architecture; copy-pasting config strings across model families is the most common silent failure.

Trainable fraction should stay under 1 to 2% for large bases unless you have evidence the task needs more capacity. Log three numbers every run: trainable params, total params, and peak optimizer state bytes (trainable params $\times$ 8 for AdamW in fp32). If optimizer state still dominates VRAM, 8-bit Adam on adapter weights or LoRA+QLoRA (4-bit base, fp16 adapters) is the next lever before buying hardware. For Stable Diffusion, caption quality beats rank: 20 well-captioned images at $r = 4$ often beats 200 duplicates at $r = 32$. Trigger tokens in captions (`sks person`) give the adapter a clean semantic hook; without them, the network may entangle subject identity with background texture. QLoRA keeps base weights in 4-bit NF4 while adapters train in fp16; memory on the frozen backbone drops roughly 4$\times$ with minor quality loss on classification. Skip LoRA on embedding and LM head unless you add vocabulary tokens; SST-2 runs typically target attention projections only. Match total train steps and effective batch against full fine-tune in **Module 09** before concluding LoRA is worse; adapter lr often needs 3 to 10$\times$ the full fine-tune rate. Save `adapter_config.json` with rank, alpha, and `target_modules` so loaders reproduce training without reading scripts. Inference with unmerged adapters adds one low-rank matmul per targeted layer; latency overhead stays under 5% for $r \leq 16$ on BERT-base classifiers. Diffusion LoRA checkpoints are often 2 to 20 MB vs multi-GB full U-Net; ship adapters to users who already have the base model cached locally. Rank-r SVD intuition: LoRA learns the top-r subspace of the fine-tune weight delta; if the task needs rank 32 but you set $r = 4$, validation loss plateaus high no matter the lr. IA3 (another PEFT method) scales activations by learned vectors with even fewer params than LoRA; try when $r = 4$ still overfits on < 500 examples. Combine LoRA with gradient checkpointing on the frozen base forward pass if activation memory from long sequences ($T = 512$) exceeds adapter optimizer cost. Dropout on adapter layers is uncommon; regularization comes from low rank and early stopping on val loss rather than adapter dropout. When serving merged LoRA weights, keep a copy of the original $\mathbf{W}_0$ checkpoint for rollback if merged model regresses on out-of-domain prompts. Hyperparameter sweep order: fix rank and targets first, then lr, then epochs; sweeping all three at once makes failures impossible to diagnose. Document base model revision (`distilbert-base-uncased` commit hash or HF revision) in training logs; adapter quality is not portable across silent base weight updates. Evaluate on a held-out prompt distribution, not just training task val set; LoRA can overfit adapter rank to SST-2 syntax while hurting general NLU probes.

14 Practice Problems

Problem 1. Count trainable parameters for LoRA on one $\mathbf{W} \in \mathbb{R}^{768\times768}$ with rank 8 and bias off.

Problem 2. Fine-tune same task with full **Module 09** setup vs LoRA; tabulate VRAM, time, and accuracy.

Problem 3. Train SD LoRA on 10 images; compare rank 4 vs 16 for overfitting signs.

Problem 4. Export merged vs adapter-only checkpoint sizes for deployment discussion.

Apply in **Lab 11**; builds on **Module 09** and **Module 10**; explanations in **Module 20** may probe adapter layers.

Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

Table 3: Certified topic map for Module 11.

Topic	Primary source	Where in this PDF
LoRA low-rank update	[3]	Eq. 2
BERT fine-tuning baseline	[2]	Section 7
QLoRA memory context	[1]	Extension section

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

References

[1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *NeurIPS*, 2023.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.

[3] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.

[4] Yinhan Liu, Myle Ott, Naman Goyal, et al. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[5] Alec Radford, Jeffrey Wu, Rewon Child, et al. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.